Autograd Deep-Dive — Problem Set (10 problems)

Drilling `r`, `requires_grad`, the graph, and `E.backward()`

These ten problems target *only* the concept we just worked through: what a tensor like `r` really is, and the exact mechanics of `E.backward()` — the graph, the chain rule, and how `.grad` gets filled. They're short. The goal isn't to write much code; it's to be able to **predict what will happen before you run it**. For most problems, write your prediction down first, *then* run and check.

How to use this sheet

1. Read each problem and **predict the answer/output first** (this is where the learning is).
2. Write a few lines to test your prediction.
3. Check against `part2_autograd_solutions.py` — run it and every problem prints `[PASS]`. I ran it; all ten pass.

Setup for every file:

```
import torch, math
torch.set_default_dtype(torch.float64)
```

Concept map

#	What it drills
1	leaf vs non-leaf, `requires_grad` propagation, when `.grad`/`grad_fn` exist
2	the graph mirrors the forward ops in reverse (`grad_fn` chain)
3	`backward()` *is* the chain rule — verify by hand
4	contributions from multiple paths **sum** (why `r` in two places works)
5	the LJ force at a new distance — autograd vs analytic
6	`.grad` **accumulates** — why `zero_grad()` exists
7	`backward()` needs a **scalar** — the `.sum()` trick
8	functional `autograd.grad` vs `.backward()`; the graph is freed after use
9	stopping gradients: `detach()` and `no_grad()`
10	`.grad` fills a whole `(N,3)` tensor → forces

Problem 1 — Anatomy of a tensor ●

Create `x = torch.tensor([2.0, 3.0], requires_grad=True)`. **Predict, before running**, the value of `x.is_leaf`, `x.requires_grad`, and `x.grad`. Then create `y = x * 2` and predict `y.is_leaf`,

`y.requires_grad`, `y.grad_fn`. Finally, create `z = torch.tensor([5.0])` (no `requires_grad`) and `w = z * 2`; predict `w.requires_grad` and `w.grad_fn`.

What you should conclude: leaves are the tensors *you* create; results of operations are non-leaves and carry a `grad_fn`. `requires_grad` is **contagious** — an output tracks gradients if *any* input does — and `.grad` is `None` until a backward pass runs.

Check: `x.is_leaf=True`, `x.grad=None`; `y.is_leaf=False`, `y.grad_fn` is not `None`; `w.requires_grad=False`, `w.grad_fn=None`.

Problem 2 — Read the graph ●●

For `r = torch.tensor([1.2], requires_grad=True)` and `E = 4.0 * (r**-12 - r**-6)`, inspect `E.grad_fn` and walk backward through `.next_functions`. **Predict the sequence of operation types** you'll find from `E` back toward `r`.

Hint: `E.grad_fn.next_functions` gives the parents; each entry is a `(fn, idx)` tuple, and constants show up as `None`.

What you should conclude: the graph is the forward computation recorded in reverse — a multiply on top, then a subtract, whose two parents are the two powers (`r**-12` and `r**-6`). This *is* the picture from the walkthrough.

Check: last op contains `"Mul"`; its non-None parent contains `"Sub"`; that node has exactly two parents containing `"Pow"`.

Problem 3 — `backward()` is the chain rule ●

Let `x = tensor(2.0, requires_grad=True)` and `f = (3*x + 1)**2`. **Compute `df/dx` by hand first** using the chain rule, then verify with `f.backward()` and `x.grad`.

Hint: outer derivative `2·(3x+1)` times inner derivative `3`.

Check: `x.grad == 42.0`.

Problem 4 — Multiple paths sum ●●

This is the crux of the LJ example (where `r` fed both `r**-12` and `r**-6`). Two cases:

(a) `y = x**2 + torch.exp(x)` at `x = 1`. Predict `dy/dx`, then check. (b) `y = (x**2) * (x + 1)` at `x = 2`. Here `x` appears in **two factors**. Predict `dy/dx`, then check.

What you should conclude: when a variable influences the output through more than one route, backward **adds up** the contribution from each route. (In (b) that's exactly the product rule — and the product rule is just "sum over the two paths.")

Check: (a) `2 + e ≈ 4.7183`; (b) `2x(x+1) + x² = 16`.

Problem 5 — The LJ force at a new distance ●●

For LJ energy `E = 4(r⁻¹² − r⁻⁶)` at **`r = 1.15`** (a value we haven't used), compute `dE/dr` by hand (`−48 r⁻¹³ + 24 r⁻⁷`), then get it from `E.backward()` → `r.grad`. Then form the force `= −dE/dr`.

Why: this is the whole point — `r.grad` *is* the physics, and you should be able to predict it.

Check: `dE/dr ≈ 1.2211`; `force ≈ −1.2211 eV/Å`. (Positive `dE/dr` because 1.15 Å is just past the LJ minimum at ≈1.122 Å, so the pair still attracts.)

Problem 6 — `.grad` accumulates ●

With `x = tensor([1.0, 2.0], requires_grad=True)`, call `(x**2).sum().backward()` **twice in a row without zeroing**. Predict `x.grad` after each call. Then `x.grad.zero_()` and do it once more.

What you should conclude: backward *adds* into `.grad`; it doesn't overwrite. That's why every training loop starts with `zero_grad()`.

Check: `[2, 4]` → `[4, 8]` → (after zero) `[2, 4]`.

Problem 7 — `backward()` wants a scalar ●●

Let `x = tensor([1.0, 2.0, 3.0], requires_grad=True)` and `y = x**2` (a **vector**). Try `y.backward()` and predict what happens. Then make it work **two** ways: (1) `y.sum().backward()`, and (2) `y.backward(gradient=torch.ones_like(y))`.

What you should conclude: `backward()` needs a single number to start from (the seed `dOut/dOut = 1`). For a vector you must either reduce it to a scalar (the `.sum()` trick we use for batches) or supply the seed vector yourself.

Check: the bare call raises `RuntimeError`; both fixes give `x.grad == 2x == [2, 4, 6]`.

Problem 8 — Functional grad, and the freed graph ●●

For `r = tensor([1.2], requires_grad=True)`, `E = 4(r⁻¹² − r⁻⁶)`: get the derivative with `torch.autograd.grad(E, r, retain_graph=True)` and compare it to what `E.backward()` puts in `r.grad`. Then, on a **fresh** `E`, call `.backward()` **twice** and predict the second call's behavior.

What you should conclude: `autograd.grad(...)` *returns* the gradient (what MLIP force code uses); `.backward()` *stores* it in `.grad` — same number. And PyTorch **frees the graph after a backward pass** to save memory, so a second backward errors unless you asked to `retain_graph`.

Check: the two derivatives are equal; the second bare `.backward()` raises `RuntimeError`.

Problem 9 — Stopping the gradient ●●

(a) `y = x**2 + x.detach()**3` at `x = 2`. Predict `dy/dx`. (b) Inside `with torch.no_grad():`, compute `z = x * 5`; predict `z.requires_grad` and `z.grad_fn`.

What you should conclude: `detach()` snips a branch out of the graph, so it contributes **zero** to the gradient; `no_grad()` builds **no** graph at all. Both matter later — e.g. a distillation *teacher's* labels are `detach()`ed so no gradient flows into the teacher.

Check: (a) `x.grad == 4` (only the `x²` term; the cubed term is invisible — it would be `16` if tracked); (b) `z.requires_grad=False`, `z.grad_fn=None`.

Problem 10 — `.grad` for a whole structure → forces ●●

Build a 4-atom LJ energy as a function of an `(N, 3)` positions tensor (`requires_grad=True`), call `E.backward()`, and set `forces = -pos.grad`. Predict the **shape** of `forces` and the value of `forces.sum(dim=0)`.

What you should conclude: `.grad` always has the same shape as the tensor it belongs to, so one `backward()` gives you the force on every atom at once — and the net force sums to ~0, which is translational invariance showing up as a conservation law.

Check: `forces.shape == (4, 3)`; `forces.sum(dim=0) ≈ [0, 0, 0]`.

The one-paragraph summary you should be able to write after this

> `r` is a leaf tensor flagged `requires_grad=True`, which makes PyTorch record every operation that uses it into a graph whose nodes (`grad_fn`) are the reverse of the forward ops. `E.backward()` seeds `dE/dE = 1` and walks that graph backward, multiplying local derivatives (chain rule) and **summing over every path** a variable takes, depositing the total into each leaf's `.grad` (which **accumulates**, needs a **scalar** to start, and is **freed** afterward unless retained). `detach()`/`no_grad()` keep parts of the computation out of the graph.

If you can write that from memory, this concept is concrete. Then the `create_graph=True` walkthrough (training *through* a force) is the natural next step — say the word.